DA2304 — Introduction to Computer Systems

Tier	A
Phase (order)	3
Do this first	DA1300
Estimated effort	12 weeks
Unlocks	DA3408, DA5402, ML systems work

Roadmap

1. Logic gates to ALU and CPU (nand2tetris style).
2. Assembly, machine code, intro to compilers and OS.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Foundations to Computer Systems Design (77 videos). NPTEL *Foundations to Computer Systems Design* (V. Kamakoti, IIT Madras): from gates to a working computer, nand2tetris style.

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- cmu 15-213/18-213/15-513: Introduction to Computer Systems, Fall 2017 (29 videos). CMU 15-213 *Introduction to Computer Systems* (Fall 2017, re-upload): data representation, assembly, caches, linking, processes, virtual memory. Do the CS:APP labs.
- Computer Sc - Computer Organization (33 videos). NPTEL *Computer Organization* (nptelhrd, IIT Madras).
- System Commands (33 videos). IIT Madras BS *System Commands*: Linux shell, the practical side.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–6	CMOS, gates, HDL simulator. Do nand2tetris Project 1.
2	7–13	Number systems, adders, the HACK ALU. Project 2.
3	14–22	Sequential logic, memory, PC, memory-mapped I/O. Projects 3 and 4 (machine language).
4	23–28	HACK CPU micro-architecture and the full computer. Project 5: build the CPU.
5	29–34	Assembler. Project 6: write the assembler in Python/C.

Week	Lectures	What to do
6-7	35–47	Virtual machine I and II. Projects 7-8: VM translator.
8-9	48–65	Jack language + compiler front end. Projects 9-10.
10-12	66–77	Compiler back end + OS. Projects 11-12. You now built a computer from NAND to a compiled program.

▼ **All lectures in *Foundations to Computer Systems Design* (77 videos)**

1. Module 1.1: Introduction to the Course
2. Module 1.2: CMOS Transistors and Gates
3. Module 1.3: Basic Gates
4. Module 1.4: Building Gates Using Simulator
5. Module 1.5: Hierarchical Design and Verification
6. Module 1.6: Building Blocks of a Digital Computer
7. Module 2.1: Binary Number Systems
8. Module 2.2: Signed Number Systems
9. Module 2.3: Two's Complement Number System
10. Module 2.4: Binary Adder Circuits
11. Module 2.5: Building the ALU of HACK
12. Module 2.6: HACK ALU Functionality
13. Module P1: Tips for Project P01
14. Module 3.1: Sequential Logic Design
15. Module 3.2: Latches and Flipflops
16. Module 3.3: The Memory Hierarchy
17. Module 3.4: Design of Program Counter
18. Module 3.5: Introduction to Computer Organization: The HACK Instruction Set Architecture (ISA)
19. Module 3.6 Memory Mapped I/O
20. Module P2: Tips for Projects P02 and P03
21. Module P3: Tips for Project 04
22. Module P4: Tips for Project 04
23. Module 4.1: Introduction to Computer Architecture
24. Module 4.2: The HACK Microarchitecture
25. Module 4.3: The HACK CPU - A Deep Dive: Part 1
26. Module 4.4: The HACK CPU - A Deep Dive: Part 2
27. Module 4.5: The Data Memory
28. Module 4.6: The HACK Computer
29. Module 5.1: The Assembler Construction
30. Module 5.2: Understanding the Working of Assembler
31. Module 5.3: Assembler: Symbol Table Construction
32. Module 5.4: Assembler: Pass 1
33. Module 5.5: Assembler: Pass 2

34. Module 5.6: Project 06 - Demonstration
35. Module 6.1: Virtual Machines - What and Why?
36. Module 6.2: The VM Instruction Set Architecture
37. Module 6.3: The execution of a VM Program
38. Module 6.4: How powerful is the VM?
39. Module P5: Project 07: VM ISA to HACK Mnemonic Translation
40. Module 6.6: Project 07: Demo
41. Module 6.5: Deep Understanding of VM ISA using VM Emulator
42. Module 7.1: Virtual Machine II - Program flow commands and Introduction to Function Calls
43. Module 7.2: Implementation of Function Call
44. Module 7.3: Working of the Virtual Machine
45. Module 7.4: Project 08: Translation of Program Flow and Function Call to HACK Mnemonic
46. Module 7.5: Handling Static Variables
47. Module 7.6: Project 08: Implementation tips in a Nut Shell
48. Module 8.1: Introduction to The JACK Programming Language
49. Module 8.2: Project 09: Basic Steps
50. Module 8.3: Understanding Syntax of JACK using Examples
51. Module 8.4: Project 09: More Examples
52. Module 8.5: The JACK Syntax - Language Specification
53. Module 8.6: Application Development using JACK
54. Module 9.1: JACK Compiler: Lexical Analysis - Tokenization
55. Module 9.2: Project 10: Compiler for JACK - Part-1 Demo
56. Module 9.3: The JACK Grammar
57. Module 9.4: Compiler for JACK: Parsing the JACK Program
58. Module 9.5: Project 10: Part 2: The Token Analyzer
59. Module 9.6: Project 10: Part 3: Testing the Correctness
60. Mod 10 1
61. Mod 10 2
62. Mod 10 3
63. Mod 10 4
64. Mod 10 5
65. Mod 10 6
66. Module 11.1: The Jack Compiler Backend: An XML guided approach
67. Module 11.2: The Jack Compiler Backend: Populating the Class and Subroutine Symbol tables.
68. Module 11.3: The Jack Compiler Backend: Code Generation - 1
69. Module 11.4: The Jack Compiler Backend: Code Generation - 2
70. Module 11.5: The Jack Compiler Backend: Code Generation - 3
71. Module 11.6: The Jack Compiler Backend: Code Generation - 4
72. Module 12.1: Jack Compiler: Code Generation - 5
73. Module 12.2: Jack Compiler: Code Generation - 6
74. Module 12.3: Jack Compiler: Code Generation - 7

75. Module 12.4: Understand the Operating System - Compiler Interactions
76. Module 12.5: Project - 12: One sample journey from Jack to Hack
77. Module 12.6: Concluding Remarks

Books, notes and websites

- nand2tetris.org (the course text) — free Coursera Parts I and II
- Book: Patterson & Hennessy, RISC-V edition; Petzold, *Code*
- ✅ NPTEL *Foundations to Computer Systems Design* (Kamakoti)
- CMU 15-213 *CS:APP*

Practice / proof of understanding

Build the Hack computer in nand2tetris projects 1-6, then an assembler.

What your IITM course sheet says

From the PDF: DA2304

Official description: Modern systems consist of multiple entities such as basic logic gates, which form the building blocks of most modern hardware systems, application programs that enable the user to interact with the system, and Operating Systems, which help coordinate between users' inputs, application programs, and the underlying hardware. It is crucial in today's world to have a proper understanding of each of these components and their interactions. Improper understanding or calibration of one or more of these components can lead to suboptimal performance and, in some cases, catastrophic consequences. This course aims to provide the students with the following outcomes: • A hands-on understanding of how computer systems work by adopting a bottom-up approach. • An insight into how the different components, viz, hardware, software, compilers, and OS, interact and the various optimizations and th

Official syllabus (trimmed):

> 1. Introduction and Outline (1 week)
> 2. Basics of Digital Design. (1 week) Boolean logic, boolean arithmetic, memory, logic gates, sequential vs combinational logic, basics of RTL design - designing basic arithmetic circuits
> 3. Computer Organization and Architecture(3 weeks) Instruction Set Architecture, Various elements of a mi- croprocessor, I/O logic, Interrupts, Memory-mapped vs interrupt-driven I/O, Computer Architecture: assembling a processor, design of memory elements, optimizations
> 4. Assembly Language and Introduction to Compilers (5 weeks) Understanding machine code, writing low- level machine programs: challenges and advantages, symbolic instruction code and micro-ops, translating high-level programs to low-level machine code: grammar, parsing, syntax-checking, code-generation, linking static and dynamic libraries.
> 5. Operating Systems and Virtual Machines (3 weeks) understanding memory management, data handling, and I/O interactions such as graphics, understanding the need and challenges of Virtual Machines, Handling program emulation in VMs.

Official textbooks: 1. The Elements of Computing Systems: Building a Modern Computer from First Principles. Noam Nisan and Shimon Schocken (MIT Press) 2. Computer Organization and Design RISC-V

Edition. David Patterson. 3. Code: The Hidden Language of Computer Hardware and Software, 2nd edition, by Charles Petzold, published by Pearson Education

Official references: 1. The Elements of Computing Systems: Building a Modern Computer from First Principles. Noam Nisan and Shimon Schocken (MIT Press) 2. Computer Organization and Design RISC-V Edition. David Patterson. 3. Code: The Hidden Language of Computer Hardware and Software, 2nd edition, by Charles Petzold, published by Pearson Education